Checkpoint project: build your own server health report

One small tool. **Every skill from Week 2.**

Claude Code in VS Code | sample files | short, simple prompts

Days 6 and 7: what we learned

Day 6: read the evidence

- Paste logs, Claude finds the story
- Disk, memory, processes: df, du, free, top
- Check a command before you run it

Day 7: safe scripts and schedules

- Runbook into a script
- bash -n, --dry-run, no sudo or rm -rf
- cron: 5 time fields
- systemd, firewall, ssh hardening

Today: Step 1 uses Day 6. Steps 2 to 4 use Day 7.

Days 8 and 9: what we learned

Day 8: Git and review

- status, diff, log only look
- Commit message: type(area): summary, then why
- PR: Summary, Changes, Risk, How to test
- Spot risky changes: 0.0.0.0/0, :latest, secrets

Day 9: Claude does the Git work

- Claude Code + gh: branch, commit, push, PR
- CLAUDE.md holds the team rules
- Never push to main. Backup branch before fixing history
- You read the PR and press merge

Today: Steps 5 and 6 use Days 8 and 9.

THE PROJECT

A script that gives you a daily health report

The report it prints

```
=== Server health report ===
--- Disk ---
OK: / is 44% full
WARNING: /var/log is 91% full
OK: /data is 42% full
--- Errors in app log ---
ERROR lines: 4
Last 3 errors:
09:18:27 Database connection refused
09:18:30 Database connection refused
10:26:47 Disk write failed on /var/log
```

Why this is useful

- Real engineers check disk and errors every day
- Run it by hand, or let cron run it
- The warning shows a real problem to fix

On a real server the script runs df and reads the real log. Today we use saved sample files so it works on every laptop.

SETUP

Open the kit. Open Claude Code.

Steps

1. Unzip **health-checkpoint.zip**
2. VS Code: **File, Open Folder**
3. Open **Claude Code**
4. Use **Manual mode**: Claude asks first

What is inside

```
health-checkpoint/
  README.txt
  sample/
    df.txt      saved "df -h" disk report
    app.log     saved app log
```

For Step 5 you need gh logged in (Day 9): run **gh auth status**. No gh? Do Steps 1 to 4.

STEP 1 | DAY 6

Ask Claude: what is going on with this server?

Prompt for Claude Code

```
Read the files in the sample folder and
explain in simple words what is going on
with this server.
Do not change anything.
```

A good answer says

- /var/log is 91% full
- The log has 4 errors
- Database connection refused, then it reconnected
- A disk write failed on /var/log

Claude reads, you check. Open sample/df.txt yourself and look for 91%.

STEP 2 | DAY 7

Claude writes the script

Prompt for Claude Code

```
Write a bash script called health_report.sh. It
reads sample/df.txt and sample/app.log and
prints a short report:
1) each disk, with WARNING if it is over 80%
full
2) how many ERROR lines the log has
3) the last 3 errors
No sudo and nothing deleted. Add a short
comment on each part. Then run it.
```

What you should see

Claude creates health_report.sh and runs it. The report looks like slide 4.

- WARNING for /var/log
- ERROR lines: 4
- 3 last errors

Read the script before you let Claude run it. Does it have sudo or rm? It must not.

STEP 3 | DAY 7

Check it yourself, then ask for a review

You run (safe)

```
bash -n health_report.sh
bash health_report.sh
```

bash -n checks the syntax without running anything.

Then ask Claude

```
Review health_report.sh. Is anything risky
or hard to understand?
Do not change the file. Give me a short
list.
```

Habit: syntax check, run on sample data, review. Never test on a real server first.

STEP 4 | DAY 7

Schedule it with cron

Prompt for Claude Code

```
Write a cron line that runs health_report.sh
every day at 8 AM and saves the output to
reports/health.txt.
Explain each part of the line in simple words.
Do not install it.
```

How to read the line

```
0 8 * * *  /path/health_report.sh >
reports/health.txt
| | | | |
| | | | +-- day of week (any)
| | | +---- month (any)
| | +------ day of month (any)
| +-------- hour: 8
+---------- minute: 0
```

We only write the line today. We do not install it. Cron needs full paths, so Claude may add a cd or an absolute path.

STEP 5 | DAYS 8 AND 9

Claude does the Git work

Prompt for Claude Code

```
Create a branch called feature/health-report. Commit
health_report.sh and the cron line as two separate
commits with clear messages.
Then create a PRIVATE GitHub repo called health-
report, push, and open a pull request into main with
a short description (Summary, Changes, Risk, How to
test).
Ask me before you push. Do not merge.
```

Then check GitHub

- Private repo
- Branch with 2 commits
- PR with Summary, Changes, Risk, How to test

Read each command before you approve. Claude should ask before the push.

STEP 6 | DAY 8

Read the PR, then merge

Reviewer checklist

- Is the change small and clear?
- Any sudo, rm or password?
- Are the commit messages clear?
- Does the PR say how to test?

Then you decide

- Happy? Tell Claude: **merge it**, or click **Merge** on GitHub
- Not happy? Ask Claude to fix it first

You press merge. Claude does not decide for you.

TRUST, THEN VERIFY

Five habits when Claude runs commands

Do not	Do
Run something you have not read	Read it, or ask Claude to explain it
Test on a real server first	Use bash -n and sample data
Accept sudo or rm -rf "because it works"	Ask for the least power needed
Paste real passwords or logs	Use fake or cleaned data
Merge without reading	Read the PR first

STEP 7

Build your prompt library

Prompt for Claude Code

```
Create PROMPTS.md with the 5 prompts we used
today as templates.
Put [square brackets] where I must fill in my
own details. One line each on when to use it.
```

Why

- Keep prompts that worked
- Reuse them at work
- Fill in the [brackets]

Tip: when a prompt is too short, add Role and Format (RCTCF from Day 2).

CHECK YOURSELF

Five quick questions

1. Which command checks a script without running it?
2. What do the five fields in a cron line mean?
3. Which Git commands only look and never change files?
4. What goes in a PR description?
5. Who presses merge?

Answers: bash -n; minute, hour, day of month, month, day of week; status, diff, log; Summary, Changes, Risk, How to test; you do.

RECAP

You built a real tool with four skills

1. **Read:** Claude explains logs and disk reports
2. **Build:** a safe script you checked
3. **Schedule:** a cron line you can explain
4. **Ship:** branch, commits, PR, you merge

Take-home

Ask Claude to add one more check (for example memory or a service status) as a new PR. Share your PR link.

Day 11 preview

Claude for Docker: containers and Dockerfiles.